

XQDot4Z: productos punto cuantizados en un procesador RISC-V

Steve Andy Ildefonso Santos

Resumen—Guardar pesos de red neuronal en cuatro bits ahorra memoria, pero el procesador todavía debe desarmarlos, y la cuantización afín añade un costo fácil de pasar por alto: un código de peso no es un peso, sino un desplazamiento respecto de un punto cero que hay que restar primero. Este trabajo pregunta si esa resta merece construirse dentro de la instrucción de producto punto. Se añade una instrucción fusionada de cuatro términos a Kuntur, procesador RISC-V docente de cinco etapas, y se compara contra tres referencias bajo una política de generación de código congelada: multiplicación por software sin multiplicador, multiplicación escalar, y un producto punto empacado cuya corrección se factoriza, calculando la suma de activaciones una vez por grupo y reutilizándola entre filas. Una campaña registrada de 2280 casos simulados, todos exitosos en dos simuladores independientes, produce un mapa de condiciones y no una sola cifra de aceleración. La fusión reduce ciclos frente a ambas, pero el margen se estrecha al crecer el número de filas, porque la factorizada reparte su corrección entre más, y desaparece con punto cero cero, donde esa alternativa la omite. Cuando varía por fila, la ventaja se paga en tamaño de código: viaja como inmediato, así que cada valor distinto necesita su propio cuerpo de fila, y la variante fusionada ocupa 516 palabras de instrucción frente a 72. Parte del hueco bruto es control de bucle y no aritmética, separados midiendo un gemelo estructural. Son ciclos simulados: no tiempo físico, energía ni área.

Palabras clave—inferencia cuantizada, aritmética de baja precisión, extensiones de instrucciones, verificación de hardware

I. INTRODUCCIÓN

Una capa de red neuronal es, por debajo, una enorme cantidad de productos punto entre pesos y activaciones. La cuantización permite guardar ambos lados como enteros pequeños con un factor de escala [1], y el ahorro en almacenamiento es notable: ocho pesos de cuatro bits caben en una palabra de 32 bits. Pero ese es un ahorro de memoria, no de trabajo. El procesador todavía tiene que desarmar la palabra antes de poder multiplicar algo, y desempacar no sale gratis.

La cuantización afín añade un segundo costo, más fácil de pasar por alto. Un código de peso de cuatro bits no es el peso. Es un desplazamiento respecto de un punto cero compartido por un grupo de pesos, así que lo que la aritmética necesita de verdad es el código menos ese punto cero. Una resta no es nada. Cuatro restas en cada grupo de cada producto punto de cada fila de una capa sí son algo.

Este trabajo hace una pregunta deliberadamente estrecha sobre esa resta: ¿merece construirse dentro de la instrucción que calcula el producto punto? Esa instrucción se añade a Kuntur, un procesador RISC-V docente de cinco etapas, y se mide contra tres puntos de referencia que resuelven la

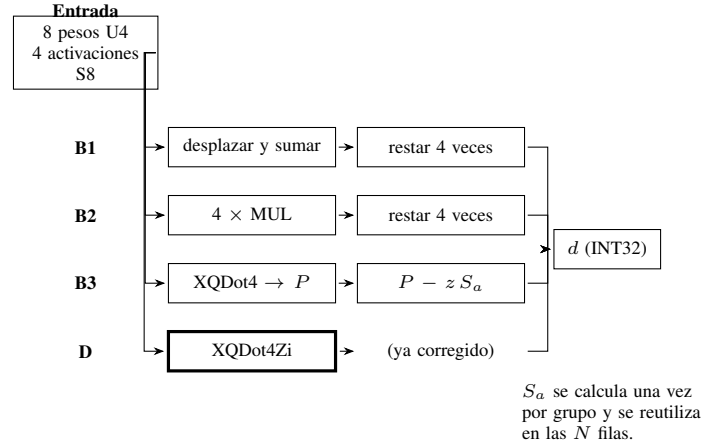


Fig. 1. Trabajo que hace cada variante sobre un mismo grupo de cuatro pesos. B1–B3 son puntos de comparación; D es el diseño en estudio. Las cuatro leen los mismos datos empacados. La diferencia no está en el producto sino en quién paga la corrección del punto cero: B1 y B2 la restan término a término, B3 la factoriza y la amortiza entre filas, y D la absorbe dentro de la propia instrucción.

misma aritmética de maneras distintas. El objetivo no es un acelerador, ni una pila de inferencia completa. Es un mapa de las condiciones bajo las cuales fusionar la corrección se paga sola, incluidas aquellas en las que claramente no lo hace.

II. ANTECEDENTES Y TRABAJO RELACIONADO

Bajo cuantización afín un código entero q representa $r = s(q - z)$, donde $s > 0$ es una escala y z un punto cero. Tomando activaciones cuyo punto cero es cero, el núcleo aritmético en estudio es

$$d = \sum_{i=0}^3 (q_{w,i} - z_w) q_{a,i}. \quad (1)$$

Los pesos se guardan como U4, enteros sin signo de cuatro bits en $[0, 15]$, y las activaciones como S8, enteros con signo de ocho bits en $[-128, 127]$. El resultado es un subtotal INT32 en $[-7680, 7680]$; escalas, sesgo y la acumulación de otros subtotales quedan fuera de la operación.

Leída al pie de la letra, (1) gasta cuatro restas antes de formar producto alguno. Sin embargo puede reordenarse como

$$d = \sum_i q_{w,i} q_{a,i} - z_w \sum_i q_{a,i}. \quad (2)$$

El primer término es un producto punto corriente sobre los códigos crudos. El segundo es el punto cero por la suma de las activaciones, y esa suma no depende de los pesos en absoluto.

TABLA I

LAS CUATRO VARIANTES. B MARCA UN PUNTO DE REFERENCIA Y D EL DISEÑO EN ESTUDIO. LAS CUATRO LEEN DATOS EMPACADOS IDÉNTICOS Y COMPARTEN UNA SOLA POLÍTICA DE GENERACIÓN DE CÓDIGO, ASÍ QUE LO QUE DIFIERE ENTRE ELLAS ES LA ARITMÉTICA, NO EL CÓDIGO QUE LA RODEA.

	Multiplicador	Producto punto	Corrección del punto cero
B1	ninguno	descomposición en bits	por término, sin multiplicación
B2	MUL escalar	un MUL por peso	por término
B3	MUL escalar	empacado, cuatro carriles	factorizada, por fila
D	no requiere	empacado, cuatro carriles	fusionada en la instrucción

En un producto matriz–vector las mismas activaciones se encuentran con todas las filas, así que puede calcularse una vez y reutilizarse en las N filas. Esa es justamente la razón por la que una comparación honesta no puede limitarse a contar las restas de (1): una implementación competente nunca las paga.

XpulpNN y Dustin son precedentes de extensiones de instrucciones de baja precisión y precisión mixta sobre RISC-V [2], [3]. Los productos punto de pocos bits no son, por tanto, una idea nueva, ni se reclaman como tal aquí. La pregunta examinada es más estrecha: dado que un producto punto empacado ya existe, ¿la corrección del punto cero pertenece dentro de la instrucción o fuera de ella?

III. PREGUNTA DE INVESTIGACIÓN Y ALCANCE

¿Bajo qué condiciones fusionar la corrección del punto cero en un producto punto U4 por S8 reduce el costo de ejecución frente a multiplicación escalar y frente a corrección factorizada, considerando la reutilización de activaciones y el costo de suministrar el punto cero?

El alcance es un solo núcleo, lote uno, y núcleos pequeños de producto punto y matriz–vector. Costo significa ciclos de un pipeline in-order simulado, reportados junto con conteos de instrucciones y tamaño de código. Síntesis, área, frecuencia y energía pertenecen a una etapa posterior con otra metodología, y nada de lo medido aquí se extrapola a ellas.

Una distinción recorre todo lo que sigue. Un punto cero puede ser constante en toda una capa (ZC), variar por fila pero conocerse al generar el código (ZS), o leerse de memoria en ejecución (ZR). La instrucción propuesta lleva el punto cero como inmediato de cuatro bits, lo que cubre los dos primeros regímenes y no el tercero. ZR queda entonces declarado fuera de alcance en vez de estimado, porque un inmediato no puede sustituir sin costo a un valor leído en ejecución.

IV. MÉTODO

A. Las cuatro variantes

Cuatro implementaciones producen las mismas salidas a partir de los mismos datos empacados. Tres son puntos de referencia, nombrados con una B inicial de *baseline*; la cuarta, D, es el diseño en estudio. La Tabla I indica qué las separa y la Fig. 1 muestra qué hace cada una con un solo grupo de cuatro pesos.

B1 no tiene multiplicador alguno y construye cada producto por descomposición en bits con máscaras acotadas. Es

contexto, no una afirmación sobre software óptimo: responde cuánto cuesta esta operación en un núcleo sin hardware de multiplicación, y su implementación se congela antes de medir para que no pueda ajustarse después de ver resultados. B2 es la versión directa con multiplicación escalar.

B3 es el competidor serio. Corre una unidad empacada de cuatro carriles sobre los códigos crudos y aplica la corrección en la forma factorizada de (2), calculando la suma de activaciones una vez por grupo y reutilizándola entre filas. D usa una unidad de cuatro carriles que resta el punto cero dentro de cada carril, así que entrega el subtotal ya corregido y nunca llega a formar una suma de activaciones.

La comparación que responde la pregunta es D contra B3. B1 y B2 acotan el espacio a su alrededor; no la resuelven.

B. Plataforma y verificación

El procesador docente original se conserva y todo el desarrollo ocurre en un clon independiente, de modo que la base no puede desplazarse bajo el experimento. Cada pieza se verifica por separado antes de combinarse.

La unidad XQDot4Z aislada supera 556 734 comparaciones por simulador contra una referencia en Python con enteros no acotados, junto con ocho controles negativos y cuatro mutaciones inyectadas que las pruebas están obligadas a detectar; el lint no necesita excepciones. La exhaustividad cubre únicamente tuplas de un término. Integrarla al pipeline como XQDot4Zi, en su forma con inmediato, añade 25 programas dirigidos en dos simuladores, comparando retiro, writeback, stores y operandos, y detectando dos mutaciones más.

La MUL escalar opcional que usa B2 es independiente de ese trabajo y devuelve los 32 bits bajos del producto [4]; no implementa ni la extensión M completa ni Zmmul. Supera 86 704 vectores por simulador y 49 programas en cuatro configuraciones. La unidad empacada detrás de B3, que omite la resta del punto cero, supera 67 910 vectores por simulador; la integración de XQDot4 supera 123 programas en ambos simuladores, cubriendo las ocho configuraciones de las tres instrucciones opcionales.

Cada hito queda fijado por SHA-256 en un archivo de estado, cada corrida se repite para mostrar que es determinista, y se inyectan mutaciones a propósito para demostrar que las pruebas pueden fallar. Dos simuladores independientes, Icarus Verilog y Verilator, deben coincidir exactamente en cada contador antes de registrar un caso como exitoso.

C. Cómo se vuelve honesta la comparación

Una comparación entre una instrucción y su ausencia es fácil de amañar, así que los grados de libertad se eliminan antes de cualquier medición. Desenrollado, asignación de registros, planificación y reutilización de la corrección quedan fijados por una sola política escrita que comparten las cuatro variantes, y esa política, junto con los generadores y las pruebas que la implementan, se congela antes de correr la campaña. La política permite explícitamente omitir trabajo que un punto cero determinado vuelve innecesario, que es como B3 termina

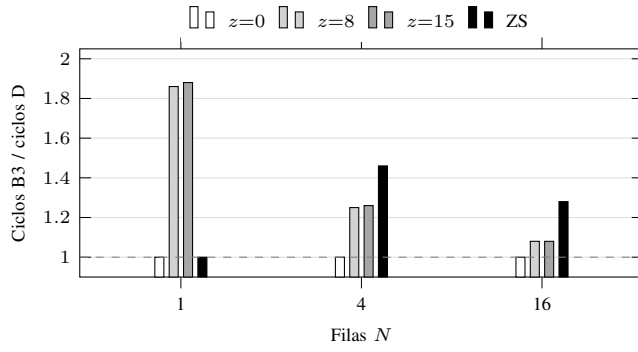


Fig. 2. Ventaja de la fusión por régimen y número de filas. La línea punteada en 1.0 marca la ausencia de ventaja. Los tres primeros grupos son puntos cero constantes; ZS los hace variar por fila. La ventaja se estrecha al crecer N porque B3 amortiza su corrección entre más filas, y en $z=0$ no existe en absoluto porque B3 la omite. En $N=1$ un punto cero variable es degenerado: una sola fila no puede tener un z que varíe, así que esas columnas coinciden necesariamente.

TABLA II
CICLOS DE B3 DIVIDIDOS ENTRE CICLOS DE D, POR RÉGIMEN Y NÚMERO DE FILAS. UN VALOR DE 1.00 SIGNIFICA QUE AMBAS VARIANTES CUESTAN EXACTAMENTE LO MISMO. CORRIDA 20260911T040655320069Z, 2280 CASOS; LOS RÉGIMENES NUNCA SE AGREGAN.

N	ZC $z=0$	ZC $z=8$	ZC $z=15$	ZS
1	1.00	1.86	1.88	1.00
4	1.00	1.25	1.26	1.46
16	1.00	1.08	1.08	1.28

eliminando su corrección por completo cuando el punto cero es cero.

La rejilla, las semillas, los perfiles de punto cero y las reglas de reporte se registran de antemano, y el registro fija las hipótesis por hash para que no puedan editarse después. Los regímenes nunca se agregan en una sola cifra, y todo caso planificado se conserva, fallos incluidos, porque un caso meramente ausente haría que una campaña incompleta pareciera terminada.

Una sutileza merece mecanismo propio. Como D lleva el punto cero en un inmediato, un punto cero que varía por fila lo obliga a emitir un cuerpo de fila por cada fila, mientras que B3 puede conservar un bucle. Parte de cualquier hueco medido es entonces control de bucle y no corrección. Para separarlos, cada variante se mide también como gemelo estructural: el mismo cuerpo de bucle replicado sin la arista de retorno. Restar el gemelo cotiza el control de bucle con exactitud, y lo que queda es atribuible a la aritmética.

V. RESULTADOS Y DISCUSIÓN

La campaña registrada corrió 2280 casos, todos exitosos en ambos simuladores y sin ninguno ausente. Los ciclos son idénticos entre las diez semillas en las 228 combinaciones de variante, forma y ajuste, y entre las dieciséis fases del perfil de punto cero variable, mientras que las salidas difieren en todas. Eso es una afirmación sobre este núcleo y este layout de datos, no una prueba de independencia frente a datos

arbitrarios, y explícitamente no se usa para justificar una rejilla más pequeña.

La Fig. 2 y la Tabla II contienen el resultado central. La ventaja de la fusión sobre B3 cae al crecer el número de filas en todo régimen donde B3 tiene corrección que amortizar, que es la tendencia que el trabajo esperaba. En $z=0$ no hay ventaja alguna: B3 omite tanto la suma de activaciones como la corrección, y las dos variantes coinciden ciclo a ciclo. En $N=1$ un punto cero variable es degenerado, ya que una sola fila no puede tener un punto cero que varíe, así que esas columnas coinciden necesariamente.

Con punto cero variable el hueco bruto mezcla dos efectos, y el gemelo estructural los separa. De los 164 ciclos que separan a las variantes en $N=16$, unos 47 pertenecen al control de bucle y 117 a la corrección misma. El cociente corregido pasa de 1.38 a 1.20, no de 1.46 a 1.28. Reportar solo la cifra bruta acreditaría a la instrucción un trabajo que hizo la estructura del bucle.

Frente a MUL escalar, D emplea entre 5.4 y 6.8 veces menos ciclos con $N=16$, con el extremo bajo en $z=0$, donde D no tiene corrección que ahorrar; B1, sin multiplicador, gasta entre 26 y 34 veces más que D.

Esa especialización forzada no es gratuita, y el costo aterriza en otra moneda. Con $N=16$ y punto cero variable, D ocupa 516 palabras de instrucción frente a 72 de B3, porque su codificación exige dieciséis cuerpos de fila separados. La ventaja en ciclos y la penalización en tamaño de código son dos caras de la misma decisión de interfaz: poner el punto cero en un inmediato es lo que vuelve gratis la corrección en ejecución y lo que hace crecer el código.

Leídas en conjunto, estas cifras responden la pregunta con una condición y no con un veredicto. La fusión ayuda más donde la corrección no puede amortizarse: pocas filas y punto cero distinto de cero. Ayuda menos, o nada, donde la alternativa puede repartir la corrección entre muchas filas o saltársela por completo. Y donde más ayuda en ciclos, con punto cero variable, es donde más cuesta en código.

A. Amenazas a la validez

El esqueleto compartido reduce el sesgo de optimización pero no demuestra optimalidad ni elimina la interacción entre software y arquitectura. La distribución sintética no representa necesariamente una red real. Con $z=0$ puede omitirse la corrección separada, lo que no demuestra igualdad de rendimiento en general. Despachar entre inmediatos sostendría un punto cero dinámico sin otra codificación, pero aquí no tiene implementación ni costo medido. Los contadores están validados estructuralmente, contra ocho programas dirigidos, tres controles negativos, cuatro mutaciones del banco detectadas y una identidad de contabilidad de ciclos comprobada en cada caso; los ciclos RTL siguen sin demostrar timing físico. La pregunta de sensibilidad queda contestada solo para el número de filas: se midió un grupo de 32 elementos, así que cómo se mueve la comparación con la cantidad de grupos y su tamaño sigue abierto, y el layout de datos congelado no alcanza hoy esas formas.

VI. CONCLUSIONES

Fusionar la corrección del punto cero en la instrucción de producto punto reduce ciclos frente a corrección factorizada y frente a multiplicación escalar, pero no de manera uniforme. El beneficio se estrecha al crecer el número de filas, desaparece cuando el punto cero es cero y, cuando el punto cero varía, se cobra de vuelta en tamaño de código. La respuesta a la pregunta no es un número sino un mapa de condiciones, y ese mapa es lo que este trabajo aporta.

Quedan fuera el régimen con puntos cero leídos en ejecución, que esta interfaz no puede expresar, y la evidencia física: área, frecuencia y energía requieren síntesis y otra metodología. El hardware adicional que exige la variante fusionada no está contabilizado en ninguna cifra reportada aquí.

REFERENCIAS

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," arXiv:1712.05877, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [2] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes," arXiv:2011.14325, 2020. [Online]. Available: <https://arxiv.org/abs/2011.14325>
- [3] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. D. Mauro, L. Benini, and D. Rossi, "Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode," arXiv:2201.08656v2, 2023. [Online]. Available: <https://arxiv.org/abs/2201.08656v2>
- [4] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: M extension, version 2.0," ISA v20260120. <https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>, 2026, accessed: 2026-09-08.